Mid-Semester Report

BY

Chaitrali Shrikant Dharmadhikari	**2024A7PS0133H**
Maddela Bhargava Krishna Reddy	**2024A8PS0349H**
Lakshya Sachdeva	**2024A8PS0468H**
Risha Sinha	**2024A8PS0485H**
Arnav Gupta	**2024A8PS0487H**

Team: Group-25

GitHub Repository: https://github.com/BinaryCoder3012/DBMS-Project.git

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF

CS F212: DATABASE SYSTEMS

(Mid-Semester Report)



BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI

HYDERABAD CAMPUS

(October, 2026)

*This report contains 5 pages, excluding the cover page.

UniGrant: Research Grant Lifecycle Management

A centralised relational database for proposals, budgets, expense approval and audit

GitHub repository: *https://github.com/BinaryCoder3012/DBMS-Project.git*

1 Problem, Users and Assumptions

UniGrant keeps the whole life of a university research grant in one MySQL database: proposal, review, award, projects, budget heads, expenses, milestones and research outputs. The rules that matter most (no overspending, at most 5 active projects per PI, no duplicate receipts) are enforced by the DBMS itself through constraints and triggers, so they hold for every client.

Role	Who	Needs	DB role
Admin	Research office	University-wide analytics, user management	admin_role
PI	Faculty	Submit proposals, log expenses, update milestones	pi_role
Grant Officer	Finance, compliance	Review proposals, approve or reject expenses	grant_officer_role
Student	Research assistants	Read assigned projects, no financial data	student_role

Core functionality. Proposal life cycle (Draft, Submitted, Under Review, Approved, Rejected); conversion of an approved proposal into a grant, projects and budget heads; expense entry and approval with automatic budget update; milestone tracking with 30-day alerts; research outputs with ordered authorship; full-text matching of funding calls to researchers; department and university analytics.

Assumptions. (A1) A user has one role and at most one department. (A2) A project has exactly one PI and at most one grant; a grant may fund several projects. (A3) Money is DECIMAL(15,2) in rupees, single currency. (A4) Only Approved expenses count towards spent_amount. (A5) MySQL has no row-level security policy, so row filtering is done with views and the application layer.

Business rules and enforcement.

ID	Rule	Enforcement
BR1	Expense cannot exceed the remaining balance of its budget head	Trigger balance_check_before_expense
BR2	Approval moves the amount into spent_amount atomically; reversal restores it	Trigger update_budget_on_approval inside a transaction
BR3	A PI may hold at most 5 active projects when proposing	Trigger active_project_limit_pi
BR4	Receipt reference is unique within a project	Trigger prevent_duplicate_expense_receipt
BR5	Dates are ordered and amounts positive; no self-supervision	CHECK constraints (11 in total)
BR6	Students see only assigned projects and no financial columns	Views Student_Project_View, Public_Project_View
BR7	Grant officers can never create users or alter schema	REVOKE CREATE USER
BR8	Expired active grants close; milestones due in 30 days notify the PI	Events grant_closure_check, milestone_30day_alert

2 Application Design

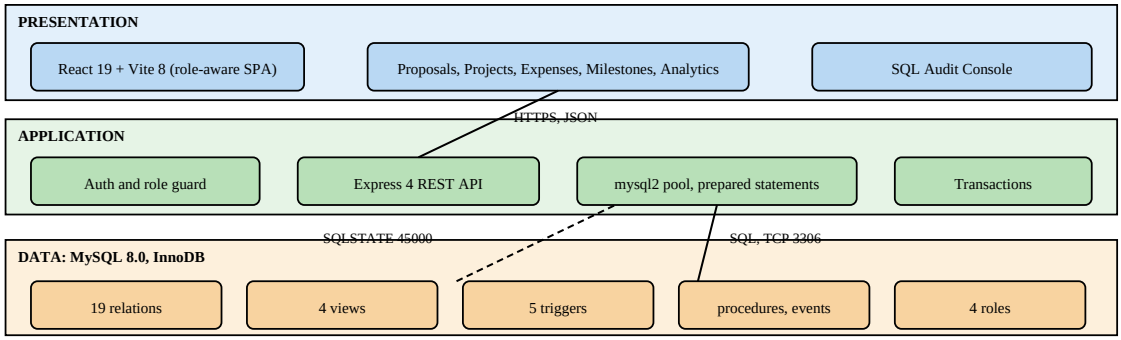


Fig. 1. Three-tier architecture: business rules live in the data tier; the API maps database errors to HTTP responses.

Item	Details
Stack	MySQL 8.0 (CHECK, roles, generated columns, CTEs, window functions); Node.js, Express 4, mysql2/promise (pooled, parameterised queries); React 19 with Vite 8; SHA-256 password hashing and role guard
W1 Award grant	Officer approves a proposal; one transaction inserts Grant_, Project and BudgetHead rows and updates the proposal; any failure rolls back (multi-table transaction)
W2 Expense	PI submits; triggers reject overspend and duplicate receipts; officer approval runs in a transaction whose AFTER UPDATE trigger updates spent_amount (integrity enforcement)
W3 Report	Full-text researcher matching, stored procedures and views for analytics (complex retrieval)
API	POST /api/auth/login, /api/proposals, /api/expenses, /api/grants/award; GET /api/{users, projects, grants, expenses, milestones}, /api/reports/department/:id; PATCH /api/expenses/:id/approve|reject

3 ER / EER Diagram (Chen Notation)

The model has 12 strong entities, 3 weak or associative entities with composite keys (Dependent, Equipment, PublicationAuthor), 3 M:N relationships with attributes (WORKS_ON, MATCHES, recursive SUPERVISES) and one specialisation of User by role (Admin, PI, Grant Officer, Student; disjoint, total), mapped to one table with a discriminator. BudgetHead and Milestone are existence-dependent on Project. Rectangles are entities, double rectangles weak entities, diamonds relationships, double diamonds identifying relationships; 1, N, M are cardinalities.

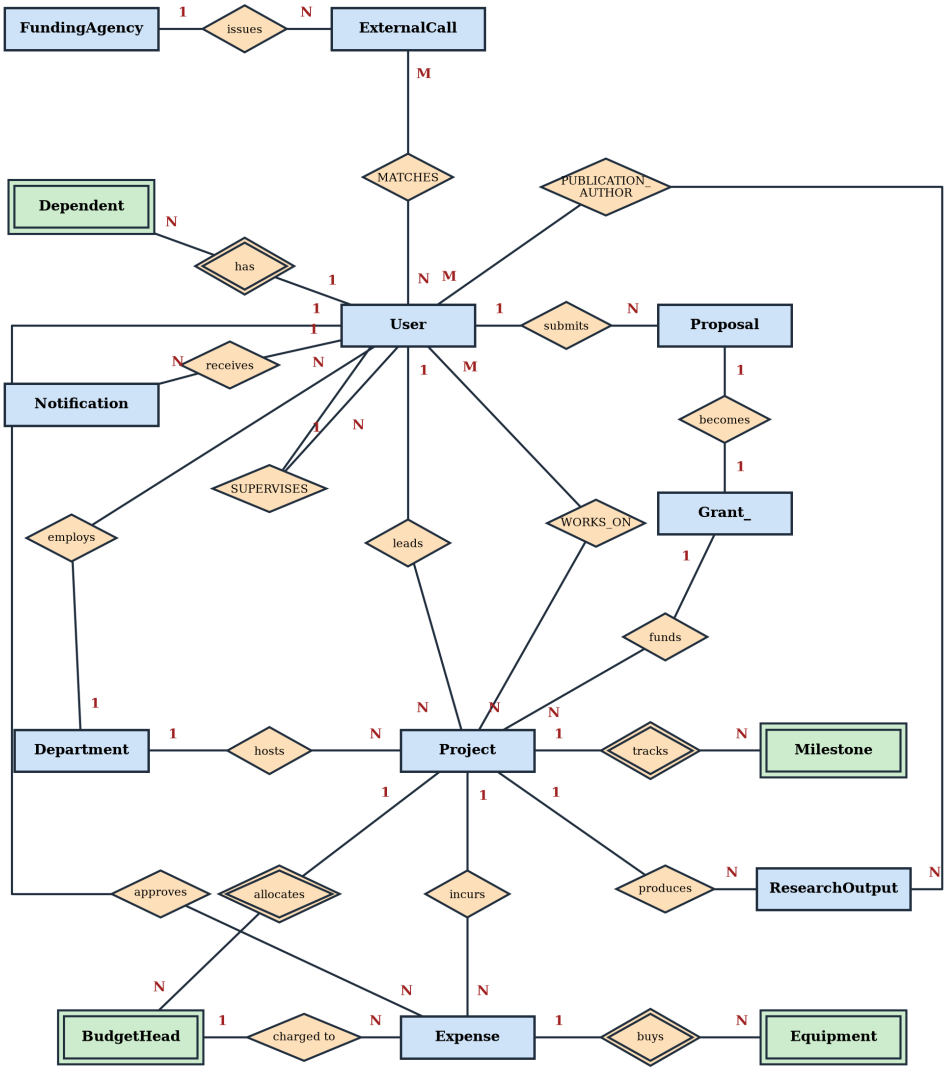


Fig. 2. Chen-style EER diagram of the complete schema.

Relationship	Participants	Card.	Participation and attributes
WORKS_ON	User, Project	M:N	role, hours/week, stipend
MATCHES	ExternalCall, User	M:N	match score 0 to 100, reason, computed at
SUPERVISES	User, User	M:N	type, start date (recursive)
PUBLICATION_AUTHOR	User, ResearchOutput	M:N	author position, contribution
becomes	Proposal, Grant_	1:0..1	Grant partial; UNIQUE(proposal_id) planned

4 Relational Schema

Mapping rules: strong entity to one relation; weak entity to a relation whose key includes the owner key; 1:N to a foreign key on the N side; M:N to a bridge relation with a composite key; specialisation to a single table with discriminator; derived value to a stored generated column. Underlined = primary key, *italic* = foreign key. 18 base relations plus the archive relation BudgetSnapshot written by a scheduled event (19 in total).

Relation	Attributes
Department	dept_id, dept_name, location, annual_budget, established_date
User	user_id, ssn (UQ), first_name, middle_initial, last_name, email (UQ), password_hash, role, phone, address, date_of_birth, hire_date, specialization, *dept_id*
FundingAgency	agency_id, agency_name, contact_person, email, phone, address, website, funding_focus_areas
ExternalCall	call_id, title, description, eligibility_criteria, deadline, max_budget, *agency_id*
Proposal	proposal_id, title, abstract, total_budget_requested, duration_months, submission_date, status, eligibility_check_passed, *pi_id*
Grant_	grant_id, grant_title, total_amount_awarded, start_date, end_date, status, funding_agency, agency_contact_info, reporting_frequency, *proposal_id*
Project	project_id, project_title, description, start_date, end_date, status, current_balance, *department_id*, *grant_id*, *pi_id*
BudgetHead	budgethead_id, *project_id*, category, allocated_amount, spent_amount, remaining_balance (generated), fiscal_year
Expense	expense_id, amount, date, description, receipt_reference, status, *budgethead_id*, *project_id*, approval_date, *approved_by*
Milestone	milestone_id, *project_id*, title, due_date, completion_date, deliverable_description, status, report_submitted, priority
ResearchOutput	output_id, *project_id*, type, title, authors, publication_date, journal_conference, doi, status
Notification	notif_id, *user_id*, message, is_read, sent_at
Dependent	*user_id*, dependent_name, relationship, date_of_birth, phone
PublicationAuthor	*output_id*, *user_id*, author_position, contribution_description
Equipment	equipment_id, *expense_id*, equipment_name, serial_number, warranty_until, location
WORKS_ON	*user_id*, *project_id*, role_in_project, hours_per_week, stipend_amount
SUPERVISES	*supervisor_id*, *supervisee_id*, relationship_type, start_date
MATCHES	*call_id*, *user_id*, match_score, match_reason, notification_sent, computed_at

Integrity constraints and referential actions

Referential actions. RESTRICT protects financial history: Proposal.pi_id, Project.pi_id, Expense.budgethead_id. SET NULL keeps optional links harmless: User.dept_id, Grant_.proposal_id, Project.department_id, Project.grant_id, Expense.approved_by. CASCADE removes existence-dependent rows (budget heads, milestones, outputs, notifications, dependents, equipment and all bridge rows).

Constraints. DECIMAL(15,2) for all money; ENUM for every status and category; UNIQUE on User.email and ssn; NOT NULL on mandatory keys and amounts; 11 CHECKs: positive requested budget, duration, grant amount, expense amount and allocation; end_date > start_date on Grant_ and Project; current_balance >= 0; spent_amount >= 0; supervisor_id != supervisee_id; match_score BETWEEN 0 AND 100; role domain. remaining_balance = allocated_amount - spent_amount is a STORED generated column. The grant relation is named Grant_ because GRANT is reserved.

Foreign key summary

Foreign key	References	ON DELETE	Foreign key	References	ON DELETE
User.dept_id	Department	SET NULL	Expense.budgethead_id	BudgetHead	RESTRICT
ExternalCall.agency_id	FundingAgency	CASCADE	Expense.project_id	Project	CASCADE
Proposal.pi_id	User	RESTRICT	Expense.approved_by	User	SET NULL
Grant_.proposal_id	Proposal	SET NULL	Milestone.project_id	Project	CASCADE
Project.department_id	Department	SET NULL	ResearchOutput.project_id	Project	CASCADE
Project.grant_id	Grant_	SET NULL	Notification.user_id	User	CASCADE
Project.pi_id	User	RESTRICT	Equipment.expense_id	Expense	CASCADE
BudgetHead.project_id	Project	CASCADE	Dependent.user_id	User	CASCADE

Bridge relations (PublicationAuthor, WORKS_ON, SUPERVISES, MATCHES) reference both parents with CASCADE.

5 Normalisation and Design Justification

All relations are in 3NF; every determinant is a candidate key except the one redundancy noted for Expense, so they are also in BCNF.

Relation	Key dependencies	Observation	NF
User	user_id → all; email → user_id; ssn → user_id	Candidate keys user_id, email, ssn	BCNF
BudgetHead	budgethead_id → all; (allocated, spent) → remaining	Derived value is a generated column	BCNF
Expense	expense_id → all	project_id also derivable via budgethead_id; kept for fast project queries	3NF
WORKS_ON, Dependent, Equipment	composite key → all non-key attributes	No partial dependency	BCNF

Documented denormalisations. (D1) Grant_.funding_agency is a snapshot of the agency name at award time. (D2) Project.current_balance is a cached roll-up refreshed in the approval transaction and checked by a test query. (D3) ResearchOutput.authors is a display string; ordered authorship lives in PublicationAuthor. Surrogate integer keys keep foreign keys narrow, STORED generated columns can be indexed, and all rules that must hold for every client live in the DBMS.

6 DBMS Features Planned

Feature	Objects and purpose
Views (4)	Public_Project_View (hides money), PI_Dashboard_View (per-PI aggregates), Admin_Analytics_View (department roll-up), Student_Project_View (row filter via WORKS_ON)
Indexes (6)	B-tree (project_title, status), (last_name, first_name), (date, project_id), (status, due_date); FULLTEXT on proposal (abstract, title) and user specialization; checked with EXPLAIN
Triggers (5)	BR1 to BR4 and auto_close_project
Transactions	Grant award (insert Grant_, Project, BudgetHead together) and expense approval
Procedures, events	generate_annual_report, get_pi_budget_summary; milestone alert, monthly budget snapshot, grant closure check
Roles (4)	admin, pi, grant_officer, student with least-privilege GRANTs; students reach data only through views
Queries (6)	Q1 department utilisation (join, GROUP BY, HAVING); Q2 above-category-average expenses (correlated subquery); Q3 top 3 projects per department (CTE, RANK() OVER); Q4 active projects with no recent milestone (NOT EXISTS); Q5 researcher matching (FULLTEXT); Q6 supervision chain (recursive CTE)

Role privileges and sample SQL

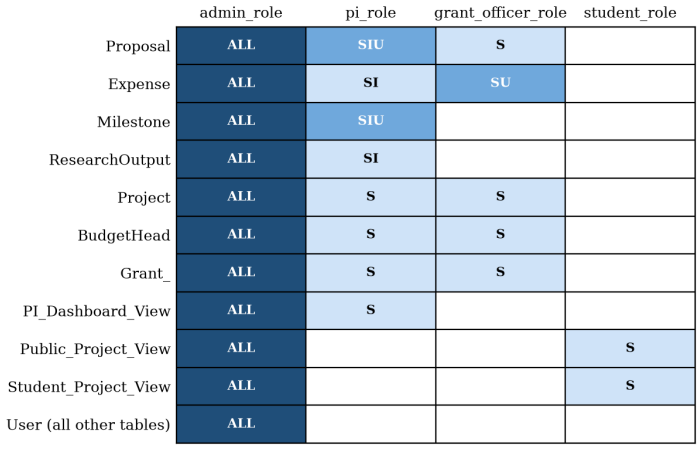


Fig. 3. Privilege matrix (S = SELECT, I = INSERT, U = UPDATE). Students reach data only through views; grant officers cannot create users.

Q3 (CTE and window function): top 3 spending projects per department.

```
WITH proj_spend AS (SELECT p.project_id, p.project_title, p.department_id, SUM(bh.spent_amount) AS spent
  FROM Project p JOIN BudgetHead bh ON bh.project_id = p.project_id GROUP BY p.project_id, p.project_title,
p.department_id)
SELECT * FROM (SELECT ps.*, RANK() OVER (PARTITION BY department_id ORDER BY spent DESC) AS rnk FROM proj_spend
ps) r WHERE rnk <= 3;
```

BR1 trigger (integrity enforcement).

```
CREATE TRIGGER balance_check_before_expense BEFORE INSERT ON Expense FOR EACH ROW BEGIN
  DECLARE avail DECIMAL(15,2);
  SELECT remaining_balance INTO avail FROM BudgetHead WHERE budgethead_id = NEW.budgethead_id;
  IF NEW.amount > avail THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'ERR-BH001: amount exceeds remaining
balance'; END IF;
END;
```

7 Progress and Team Contributions

Done: requirements, EER/ER model, relational mapping, DDL for all relations with keys and constraints, repository scaffold, this report. **In progress:** seed data (500+ rows), views and indexes, triggers and RBAC, API scaffold. **Planned:** React screens, analytics, backup notes, integration testing; final submission 15 November.

Member contributions

Member	Roll number	Contribution
Chaitrali Shrikant Dharmadhikari	2024A7PS0133H	Triggers, transactions and integrity; stored procedures and events; expense approval screen
Maddela Bhargava Krishna Reddy	2024A8PS0349H	Express API, connection pool, error mapping; RBAC (04_rbac.sql); login and user screens
Lakshya Sachdeva	2024A8PS0468H	Schema and EER modelling; relational mapping and normalisation; 01_schema_er.sql; seed data; README
Risha Sinha	2024A8PS0485H	Frontend design system and shell; Proposals, Projects, Milestones, Outputs, Grants and Departments screens
Arnav Gupta	2024A8PS0487H	Views and indexes, EXPLAIN review; queries Q1 to Q6; analytics; full-text matcher; backup notes

Responsibility matrix (lead = L, support or reviewer = s)

Area	Chaitrali	Bhargava	Lakshya	Risha	Arnav
EER and relational schema	s	s	L	s	s
Seed data and constraints	s		L		s
Views and indexes			s		L
Triggers and transactions	L	s			
Procedures and events	L				s
RBAC and authentication	s	L		s	
REST API	s	L	s		s
Frontend screens	s	s		L	s
Analytics and reports				s	L

Build schedule. Seed data and views by 20 October, triggers and RBAC by 24 October, API by 26 October, screens by 3 November, integration and testing by 10 November, final submission 15 November, demos 17 to 19 November.

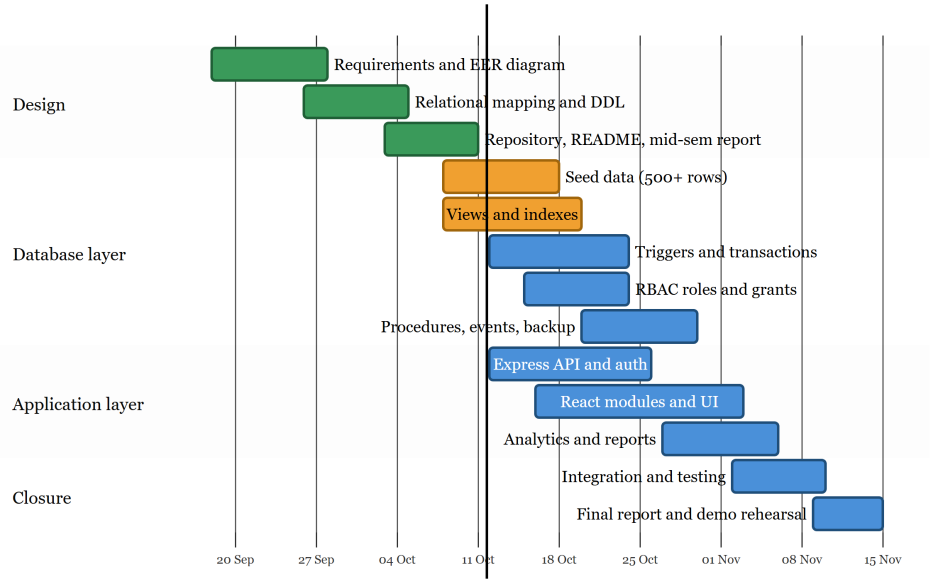


Fig. 4. Build schedule to the final demo.

Risk	Mitigation
Schema changes after the API starts cause rework in three layers	Schema freeze on 16 October; changes only through the schema owner
Trigger interactions (BR1 with BR4) cause surprising rollbacks	Test scripts with asserted SQLSTATE 45000 failures
MySQL has no native row-level security	Views plus role guard, tested per role

Collaboration. One Git branch per layer (database, API, UI); every merge into main is reviewed by a member outside that layer; each SQL file ships with a test script.